C programming

Brief introduction of topics which we will cover into this chapter are as follows

Date types

Different types of data types is used in a program which specifies memory space occupied by the variables, such as integers, floating-point numbers, characters, and Boolean values. These data types determine how the data is stored in memory and how it can be manipulated in the program. Some common data types in C include int, double and char.

Operators:

Operators in C which are symbols or keywords that are used to perform operations on data, such as arithmetic operations, comparison operations, logical operations, and bitwise operations. Arithmetic operators include addition, subtraction, multiplication, and division, while comparison operators include equal to, not equal to, greater than, and less than. Logical operators include AND, OR, and NOT, and bitwise operators include AND, OR, and Exclusive-OR.

Loops

There are three types of loops in C: for loop, while loop, and do-while loop. A for loop repeats a block of code a specific number of times, a while loop repeats a block of code while a certain condition is true, and a do-while loop repeats a block of code at least once, and then continues to repeat the code while a certain condition is true.

Arrays

An array is a collection of elements that are stored in contiguous memory locations. Arrays can be declared with a specific size, and each element in the array can be accessed using its index value.

```
int list[5];
float matrix[2][2];
```

Functions:

After this we will discuss function—which is blocks of code referred by a name that perform a specific task. Functions can be defined by the user, or they can be library functions provided by the C library. Functions can be called from any parts of the program, and they can be passed arguments and return values. Functions are often used to break down a large program into smaller, more manageable pieces.

Structure:

After this we will discuss structure in C which is a user-defined data type that allows to group different data variables together into a single unit. A structure can contain data variables such as integers, floating-point numbers, characters, and even other

structures. Structures are often used to represent complex data structures, such as linked lists and trees.

```
struct Person {
      string name;
      int age;
read();
} person1, person2;
```

C Tokens:

In C, a token is the smallest unit of a program that is meaningful to the compiler. C has several types of tokens, including identifiers, keywords, literals, operators, and punctuators.

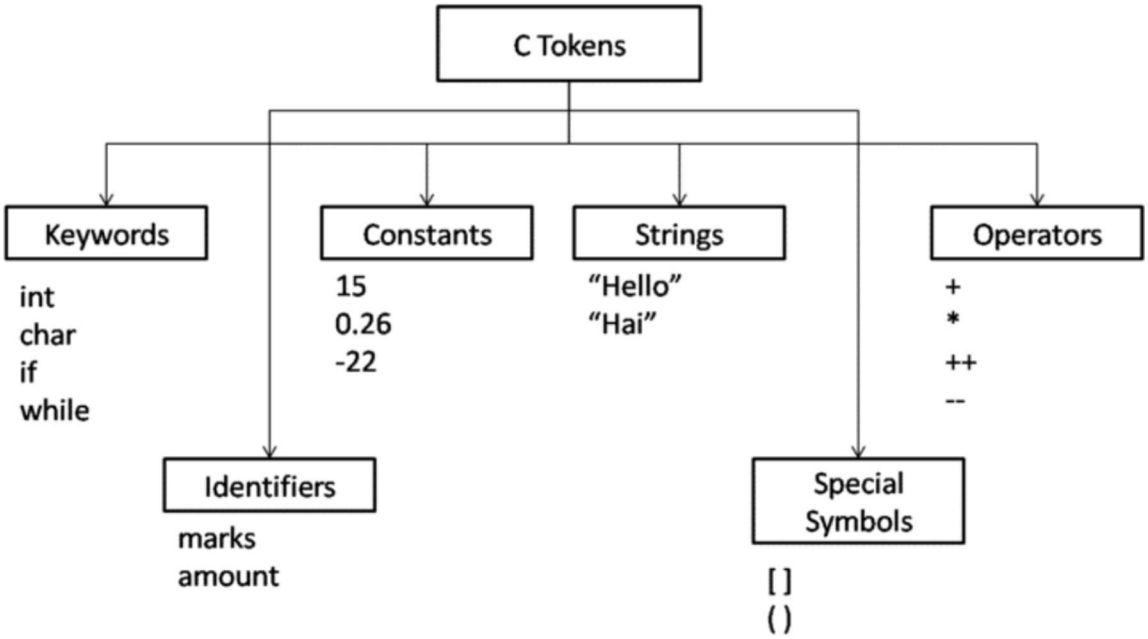


Figure 1: Tokens in c

1. **Literals/Constant:** Literals represent constant values in a program. Examples of literals include integer literals (e.g. 42, -10), floating-point literals (e.g. 3.14, -2.5e-3), and string literals (e.g. "Hello, world!").

2. **Identifiers/Variables:** An identifier is a name given to a variable, function, or other user-defined entity in a program. Identifiers must begin with a letter or underscore, and can contain letters, digits, and underscores.

3. **Keywords:** Keywords are reserved words in C that have a specific meaning to the compiler. Examples of keywords include int, double, if, for, and while.

4. **Operators:** Operators are symbols used to perform operations on variables or values. Examples of operators include arithmetic operators (e.g. +, -, *, /), comparison operators (e.g. ==, !=, <, >), and logical operators (e.g. &&, ||, !).

5. **Special characters:** Punctuators are symbols used to separate parts of a program or perform specific actions. Examples of punctuators include semicolons (;), commas (,), parentheses (()), and braces ({}).

Literals/Constant:
It is *Name of the memory Location* where value can't change during the execution of the program.

Constants refer to fixed values that the program may not alter is called **literals/constant**.

Constants can be Integer Numerals, Floating-Point Numerals, Characters, Strings and Boolean Values.

Integer constant:

- It is combination digits [0-9].
- Integer constants have no fractional parts.
- You can specify integer constants in decimal, octal, or hexadecimal form.
- They can specify signed or unsigned types and long or short types

In C, there are several types of integer constants that can be used in a program:

- Decimal integer constants: These are integer constants that are expressed in base-10 notaion. For example: 42, -10, 123456.

- Octal integer constants: These are integer constants that are expressed in base-8 notation, and are preceded by a 0 (zero). For example: 012, 077, 0765.

- Hexadecimal integer constants: These are integer constants that are expressed in base-16 notation, and are preceded by 0x or 0X. For example: 0x2A, 0xFF, 0x1A7.

- Binary integer constants: These are integer constants that are expressed in base-2 notation, and are preceded by 0b or 0B. Binary integer constants were introduced in C14. For example: 0b1010, 0B11001101.

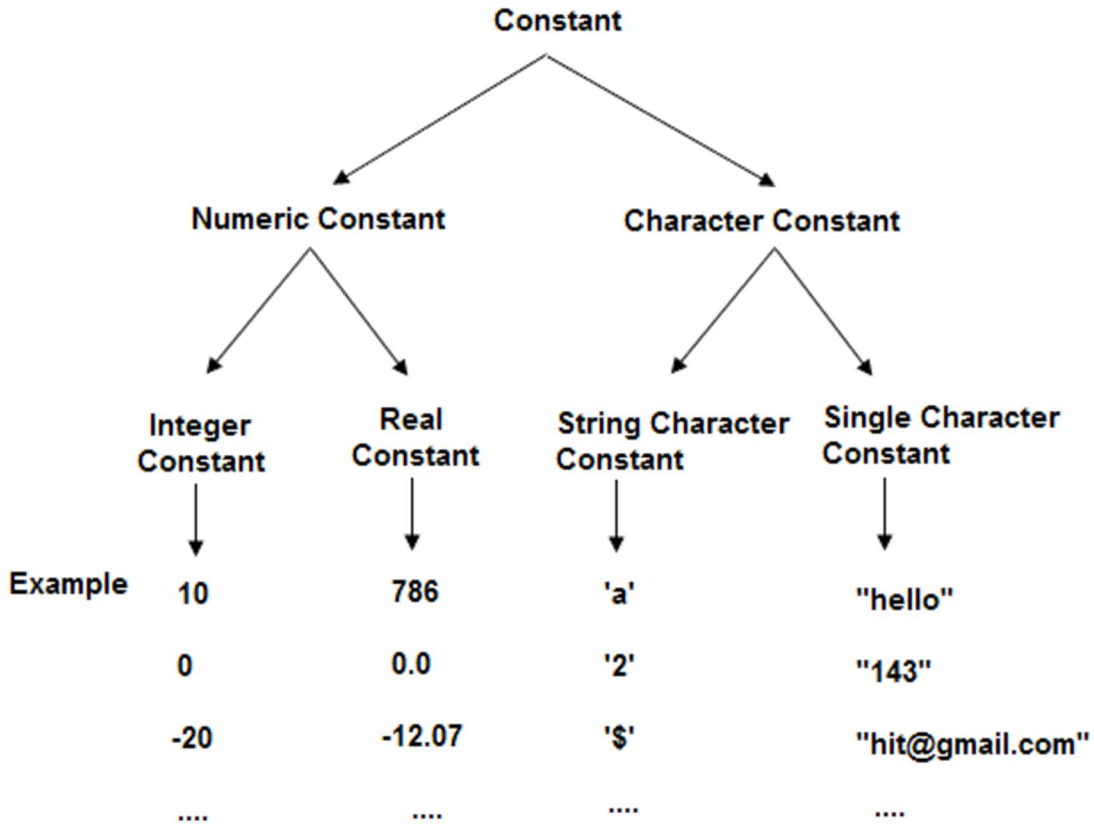


Figure 2: Types of constants

Q: Here are some examples of valid and invalid identifiers in C:

`Valid Identifiers:`

- `age`
- `_count`
- `length_of_string`
- `number_1`
- `first_name`
- `sum_of_values`
- `is_true`

`Invalid Identifiers:`

- `1number`
  `(identifiers cannot start with a digit)`

- `my-variable`
  `(identifiers cannot contain hyphens or other special characters)`
- `class`
  `(identifiers cannot be keywords in the programming language)`
- `my name`
  `(identifiers cannot contain spaces)`
- `123abc`
  `(identifiers cannot consist solely of digits)`
- `while`
  `(identifiers cannot be reserved words in the programming language)`

Note that C is a case-sensitive programming language, so age and Age would be considered two different identifiers.

Decimal Constant:
In C, a decimal constant is a type of integer constant that is expressed in base-10 notation. It consists of a sequence of decimal digits (0-9) with an optional sign (+ or -) and an optional decimal point.

Here are some examples of decimal constants in C:

```
42
-10
123456789
3.14f       //float constant
3.14  // double by default
-0.5
```

Decimal constants can be represented using various data types in C, such as float and double. *The default data type for a decimal constant is double.*

Char Constant:
In C, a char constant is a type of constant that represents a single character enclosed in single quotes. It can be a letter, a digit, a symbol, or an escape sequence.

Here are some examples of char constants in C:

- 'a' —97(ASCII Code)
- 'B' —66
- '1' —49
- '+'
- '\n' (represents a newline character)
- '\t' (represents a horizontal tab character)
- '\' (represents a backslash character)

In C, a char constant is stored as an integer value representing its ASCII code. For example, the char constant 'a' is stored as the integer value 97, which is the ASCII code

for the lowercase letter 'a'. The escape sequences in char constants represent special characters that cannot be typed directly on a keyboard, such as newline and tab.
How to create

```
const char key='A';   // key=65 will store into memory.
```

String constant:
In C, a string constant is a sequence of characters enclosed in double quotes. It is used to represent a string of characters, such as a word, a phrase, or a sentence.

Here are some examples of string constants in C:

```
"Hello, world!"
"The quick brown fox jumps over the lazy dog."
"1234567890"
"C programming is fun!"
```

String constants are stored in memory as an array of characters with a null character ('\0') at the end to indicate the end of the string. The size of a string constant includes the null character.

```
String name="Arjun";
or
char name[]="Arjun";// A   r   j   u   n  '\0'
```

To declare a string variable in C, you can use the string class, which is a part of the C standard library. Here is an example:

```
#include <stdio.h>
int main() {
    char myString[] = "Hello, world!";
    printf("%s\n", myString);
    return 0;
}
```

How to create a symbolic constant in C:

1. In C, you can create a variable as a constant using the **const** keyword. A constant variable is a variable whose value cannot be changed once it is initialized. Here is an example:

```
#include <iostream>
int main() {
    const int MY_CONST = 10;
    std::cout << MY_CONST;
    // MY_CONST = 5;  // compilation error
    return 0;
}
```

In this example, we have declared a constant integer variable called MY_CONST and initialized it with the value 10. The const keyword before the variable name indicates that this variable is a constant and its value cannot be changed. If we try to assign a new value to MY_CONST, the compiler will generate an error.

2. **Constants** can also be defined using the #define pre-processor directive*, but using const is generally preferred as it provides type safety* and can be used with more complex data types. Here is an example of defining a constant using #define:

```
#include <stdio.h>
#define MY_CONST 10 // it is always global in scope
int main() {
    printf("%d\n", MY_CONST);
    // MY_CONST = 5; // would still cause a compilation error
    return 0;
}
```

In this example, we have defined a constant integer called MY_CONST using the #define directive. This is a text substitution performed by the pre-processor, so wherever MY_CONST appears in the code, it will be replaced with the value 10. However, this approach does not provide type safety and can cause errors if used improperly. Therefore, using const is generally recommended over #define for defining constants in C.

In C, preprocessor directives can be used to define macros, which can behave like functions. These macros are evaluated by the preprocessor before the actual compilation of the code. Macros defined with the #define directive can take parameters and be used in place of functions, but they do not have type checking or the ability to return values like regular functions do.

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x)) // Macro that computes the square of a number
int main() {
  int num = 5;
  printf("Square of %d is %d\n", num, SQUARE(num)); // SQUARE macro used as a function
  printf("Square of (3 + 2) is %d\n", SQUARE(3 + 2)); // Watch out for operator precedence
  return 0;
}
```

OUTPUT
Square of 5 is 25
Square of (3 + 2) is 25

Keywords

Keywords in C are a predefined set of reserved words that have special meanings in the language and cannot be used as identifiers (variable names, function names, etc.) in a program.

Here is a list of keywords in C:

asm	double	new	switch
auto	else	operator	template
break	enum	private	this
case	extern	protected	throw
catch	float	public	try
char	for	register	typedef
class	friend	return	union
const	goto	short	unsigned
continue	if	signed	virtual
default	inline	sizeof	void
delete	int	static	volatile
do	long	struct	while

Figure 3:List of keywords in C:

Identifiers / Variables:

- A variable is a named memory location that is used to store a value of a specific data type. This value can be change during the execution.
- Variable is user-defined names that are used to represent various elements in a C program such as variables, functions, classes, etc.
- By default all the variable are created in Primary Memory.
- Occupied Memory size by variables are compiler depended.

C supports many different data types for variables, including:

int: integers (whole numbers)
float: single-precision floating-point numbers
double: double-precision floating-point numbers
char: single characters
bool: boolean values (true or false)
long: larger integers
short: smaller integers

How to create Variables/ Definition of variables:
In C, variables are created by declaring them with a specific data type and an optional initial value. The syntax for creating a variable in C is:
Syntax:
data_type variable_name = initial_value;

```
int a=34;
float b;// Garbage value into b;
Garbage value means—Default value at  that memory location.
Here a and b are variables
Char signal='R';
```

Naming Rules to create variables:

In C, variables are one of the most fundamental building blocks of any program. They allow programs to store and manipulate data as needed. However, there are certain rules that must be followed when creating variables in C. Here are some of the key rules:

1. Variable names must begin with a letter or an underscore character (_), and can be followed by letters, digits, or underscore characters.

2. Variable names are case-sensitive. For example, myVar and myvar are considered two different variables.

3. Variable names cannot be the same as a C keyword. For example, you cannot use int, float, or while as a variable name.

4. Variable names should be meaningful and descriptive, so that the purpose of the variable is clear.

5. Variables must be declared before they are used. This means that you must specify the data type and name of a variable before you can assign a value to it or use it in an expression.

6. Variables can only store values that are compatible with their data type. For example, you cannot assign a string value to an integer variable.

7. Variables can be initialized with an initial value at the time of declaration, or later on in the program using an assignment statement.

8. Variable scope refers to the area of the program where the variable can be accessed. A variable declared inside a function can only be accessed within that function, while a variable declared outside of any function (i.e. at the global scope) can be accessed from any part of the program.

9. Variables can be modified or reassigned at any time during the program's execution.

By following these rules, you can create well-formed and effective variables in your C programs.

Question: Difference between x and ‘x’ In C program.

```
char a= 'a';
a='w';
```

a—variables are used to store values that can be changed during the execution of a program, while character **‘a’** constants are used to represent fixed ASCII values (97) that cannot be changed.

Operators:

Operators in C are symbols that are used to perform various operations on one or more operands. C supports a wide range of operators, including arithmetic, relational, logical, bitwise, and assignment operators, among others. Here are some of the most commonly used operators in C:

1. **Arithmetic Operators:** These are used to perform basic mathematical operations such as addition (+), subtraction (-), multiplication (*), division (/), and modulus (%).

2. **Relational Operators:** These are used to compare two values and return a Boolean value (true or false) based on the result. The relational operators include less than (<), greater than (>), equal to (==), not equal to (!=), less than or equal to (<=), and greater than or equal to (>=).

3. **Logical Operators:** These are used to perform logical operations on Boolean values. The logical operators include logical AND (&&), logical OR (||), and logical NOT (!).

4. **Bitwise Operators:** These are used to perform operations on individual bits of values. The bitwise operators include bitwise AND (&), bitwise OR (|), bitwise NOT (~), bitwise XOR (^), left shift (<<), and right shift (>>).

5. **Assignment Operators:** These are used to assign a value to a variable. The basic assignment operator is the equals sign (=), but there are also shorthand assignment operators such as addition and assignment (+=), subtraction and assignment (-=), and so on.

6. **Increment and Decrement Operators:** These are used to increment or decrement the value of a variable by 1. The increment operator is written as ++ and the decrement operator is written as --.

7. **Ternary Operator:** This is a shorthand way of writing an if-else statement in a single line. The syntax is: condition ? expression1 : expression2. If the condition is true, expression1 is evaluated, otherwise expression2 is evaluated.

Already we have learn all these operator in c programming so we are briefing all operators.

Arithmetic Operators

There are following arithmetic operators supported by C language Assume variable A holds 10 and variable B holds 20, then −

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator, increases integer value by one	A++ will give 11
--	Decrement operator, decreases integer value by one	A-- will give 9

Figure 4: Arithmetic operators

Relational Operators

There are following relational operators supported by C language Assume variable A holds 10 and variable B holds 20, then –

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Figure 5: Relational Operators

Logical Operators

There are following logical operators supported by C language. Assume variable A holds 1 and variable B holds 0, then –

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	(A && B) is false.
||	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.	(A || B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true, then Logical NOT operator will make false.	!(A && B) is true.

Figure 6: Logical Operators

Bitwise Operators

Bitwise operator works on bits and perform bit-by-bit operation. The truth tables for &, |, and ^ are as follows –

p	q	p & q	p | q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Figure 7: Bitwise Operators

Assume if A = 60; and B = 13; now in binary format they will be as follows –

```
A = 0011 1100
B = 0000 1101
-----------------
A&B = 0000 1100
A|B =       0011 1101
A^B =       0011 0001
~A  =       1100 0011
```

Assume variable A holds 60 and variable B holds 13, then –

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) will give 61 which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 0000 1111

Figure 8: Bitwise Operators

Assignment Operators

There are following assignment operators supported by C language –

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand.	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
|=	Bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

Figure 9:Assignment Operators

Misc Operators

The following table lists some other operators that C supports.

sizeof(): operator returns the size of a variable.

```
//For example,
int a;
int x=sizeof(a);
// where 'a' is integer, and will return 2.
```

1. Ternary operator:

```
Expression? X : Y;
```

Conditional operator (?). If Condition is true then it returns value of X otherwise returns value of Y.

```
#include <iostream>
using namespace std;

int main () {
   // Local variable declaration:
   int x, y = 10;

   x = (y < 10) ? 30 : 40;
   cout << "value of x: " << x ;

   return 0;
}
Output:
value of x:40
```

Precedence Rules:
In computer programming, precedence rules determine the order in which operations are evaluated within an expression.

Here are some common precedence rules in programming:

- **Parentheses:** Operations inside parentheses are always evaluated first.
- **Exponentiation:** Exponentiation, such as raising a number to a power, has higher precedence than multiplication, division, and addition.
- **Multiplication and Division:** Multiplication and division have higher precedence than addition and subtraction.
- **Addition and Subtraction:** Addition and subtraction have the lowest precedence among arithmetic operations.
- **Comparison Operators:** Comparison operators, such as greater than and less than, are evaluated after arithmetic operations.
- **Logical Operators:** Logical operators, such as AND and OR, have lower precedence than comparison operators.
- **Assignment Operators:** Assignment operators, such as = and +=, have the lowest precedence and are evaluated last.

Precedence and associativity/ Evolution of an expression:

It's important to understand precedence rules when writing complex expressions, as they can affect the outcome of the program. You can use parentheses to explicitly specify the order of evaluation if you want to override the default precedence rules.

Precedence and associativity rules are two important concepts in computer programming that help determine the order in which operators are evaluated within an expression.

Precedence rules determine the order of operations based on the operators used in the expression. Each operator has a precedence level, which determines its order of evaluation. Operators with higher precedence are evaluated before operators with lower precedence. For example, in the expression "2 + 3 * 4", multiplication has higher precedence than addition, so the expression is evaluated as "2 + (3 * 4)".

Associativity rules come into play when multiple operators with the same precedence level are used in the same expression. Associativity rules determine whether the operators are evaluated from left to right or from right to left. For example, in the expression "6 / 3 / 2", division has left associativity, so the expression is evaluated as "(6 / 3) / 2". If the division operator had right associativity, the expression would be evaluated as "6 / (3 / 2)".

Precedence	Operator	Description	Associativity
1	: :	Scope resolution	Left to right
2	a++ a— type() type{} a() a[] . ->	Postfix increment and decrement Function cast Function call Subscript Member access	Left to right
3	++a - -a +a -a ! ~ (type) *a &a sizeof co_wait new new[ ] delete delete[ ]	Prefix increment and decrement Unary plus and minus Logical and bitwise NOT C-Style cast Dereference Address of Size-of Await expression Dynamic memory allocation Dynamic memory deallocation	Right to left
4	.* ->*	Pointer to member	Left to right
5	a*b a/b a%b	Multiplication, division, remainder	
6	a+b a-b	Addition , subraction	
7	<< >>	Bitwise left and right shift operators	
8	<= >	Three way comparision	
9	< <= > >=	Relational operators	
10	== !=	Equality and not equality check operators	
11	&	Bitwise AND	
12	^	Bitwise XOR	
13	|	Bitwise OR	
14	&&	Logical AND	
15	||	Logical OR	
16	a ? b:c throw co_yield = += -= *= /= %= <<= >>= &= ^= |=	Ternary conditional operator throw operator yield-expression Direct assignment Compound assignment by sum, difference Compound assignment by product,quotient,remainder Compound assignment by bitwise left and right shift Compound assignment bY bitwise AND, XOR, OR	Right to left
17	,	comma	Left to right

Figure 10: Precedence and associativity of various operators

Here are five examples of how an expression can evolve:

1). Original expression: 2 + 3 * 4
Evolved expression: 2 + 12
Explanation: The multiplication operator has a higher precedence than the addition operator, so 3 * 4 is evaluated first. The result of that operation is 12, which is then added to 2.

2). Original expression: (2 + 3) * 4
Evolved expression: 5 * 4
Explanation: The parentheses are evaluated first, resulting in 2 + 3 = 5. Then the multiplication operator is evaluated, resulting in 5 * 4 = 20.

2. Original expression: 3 * (4 + 2) / 2
   Evolved expression: 18 / 2
   Explanation: The parentheses are evaluated first, resulting in 4 + 2 = 6. Then the multiplication operator is evaluated, resulting in 3 * 6 = 18. Finally, the division operator is evaluated, resulting in 18 / 2 = 9.

3. Original expression: 5 + 3 > 7 - 2
   Evolved expression: true
   Explanation: The subtraction operator is evaluated first, resulting in 7 - 2 = 5. Then the addition operator is evaluated, resulting in 5 + 3 = 8. Finally, the greater-than operator is evaluated, resulting in the Boolean value true.

4. Original expression: x = 2 * y + 3
   Evolved expression: x = 7 (assuming y = 2)
   Explanation: The multiplication operator is evaluated first, resulting in 2 * 2 = 4. Then the addition operator is evaluated, resulting in 4 + 3 = 7. Finally, the assignment operator sets the value of x to 7

```
int x = 1, y = 2, z;
z = (x++, y++, x + y); // the comma operator
```

In this example, the comma operator is used to evaluate three expressions and return the value of the last one. Here's what happens:

- The x++ expression is evaluated first. This increments the value of x from 1 to 2, and the result of the expression is discarded.
- The y++ expression is evaluated next. This increments the value of y from 2 to 3, and the result of the expression is discarded.
- The x + y expression is evaluated last. This adds the values of x and y, which are now 2 and 3 respectively, and returns the result, which is 5.
- The value of the entire expression (x++, y++, x + y) is 5, so that value is assigned to z.

C Data Types

A data type is a classification of data that determines what type of values can be stored in a particular variable, how those values can be manipulated, and the amount of memory that is required to store the values. Different programming languages provide different data types to represent different kinds of data.

Common data types include:

Primitive Built-in Types

1. **Integer:** used to represent whole numbers, such as 5, -10, and 0. Examples of integer data types include int, short, long, and long long in C/C.

2. **Floating-point:** used to represent real numbers with decimal points, such as 3.14 and -0.5. Examples of floating-point data types include float, double, and long double in C/C.

3. **Boolean:** used to represent true or false values. In C/C, the bool data type is used for Boolean values.

4. **Character:** used to represent single characters, such as 'a', '7', and '@'. Examples of character data types include char and wchar_t in C/C.

5. **Pointer:** used to store memory addresses, which are used to access and manipulate data stored in memory.

Secondary Built-in Types

6. **String:** used to represent a sequence of characters. In C/C, a string is typically represented as an array of characters, often terminated by the null character '\0'.

7. **Array:** used to store a fixed-size collection of values of the same data type.

8. **Structure:** used to group together values of different data types into a single entity.

9. **Enumerated type:** used to define a set of named values, such as the days of the week or the colours of the rainbow.

In C, data types are used to specify the type of data that a variable can hold. C supports a variety of data types, each with its own characteristics and uses. Here is a detailed explanation of C data types with examples.

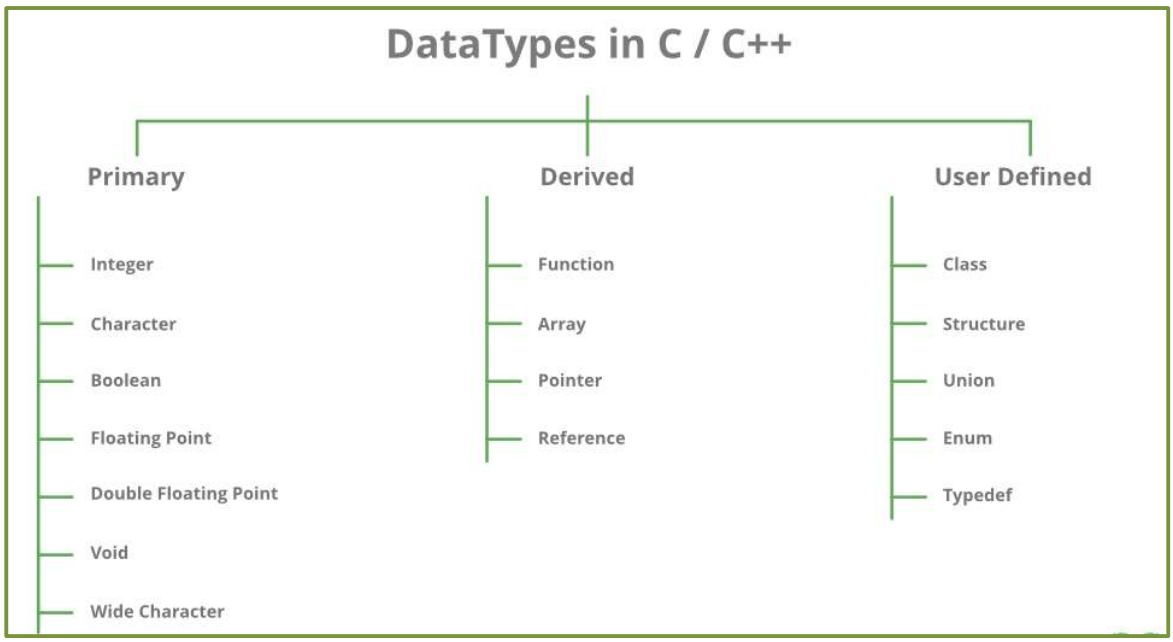


Figure 11: Data types in c

The following table shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

Data Type	Data Size (bytes)	Minimum value	Maximum value
char	1	-128	127
short	2	-327868	327867
int	2 (16 bit compiler)	-327868	327867
	4 (32 bit compiler)	-2147483648	2147483647
long	4	-2147483648	2147483647
float	4	-3.4E-38	3.4E+38
double	8	-1.7E-308	1.7E+308
long double	10	-3.4E-4932	1.1E+4932

Figure 12: Data types and it's memory

Variable Definition:
To define a variable that can hold a number in C, you need to choose the appropriate data type based on the range and precision of the numbers you want to work with. Here are some examples:
int variable_name;
float variable_name;
double variable_name;
long double variable_name;

Variable Initialization:
We can initialize a variable when you define it by assigning an initial value to it.
Data_type variable_name = constantValue ;

int math_marks = 42; //or int math_marks(42);
cout<<math_marks;

char and String initialization in C :
To initialize a char variable with a single character, you can use single quotes ' ' to enclose the character. For example:

```
char myChar = 'a';
```

String initialization:

To initialize a string variable with a string of characters, you can use double quotes " " to enclose the characters. For example:
string myString = "Hello, world!";

Type Conversion in C :

Type conversion, also known as **typecasting**, is the process of converting the data type of a variable or expression from one data type to another. In C, there are several ways to perform type conversion:

Implicit Type Conversion:

This happens automatically by the compiler when the data types of two expressions in an operation are not the same. The compiler automatically converts one of the expressions to the data type of the other expression before the operation is performed.

```
int a = 5;
float b = 2.5;

// Implicit conversion of int to float before the addition
float c = a + b;

// Implicit conversion of float to double before the assignment
double d = c;
```

Regenerate response

Explicit Type Conversion: This is done manually by the programmer by using typecasting operators. The syntax for explicit typecasting is:

(Datatype) expression;
Or
Datatype (expression);

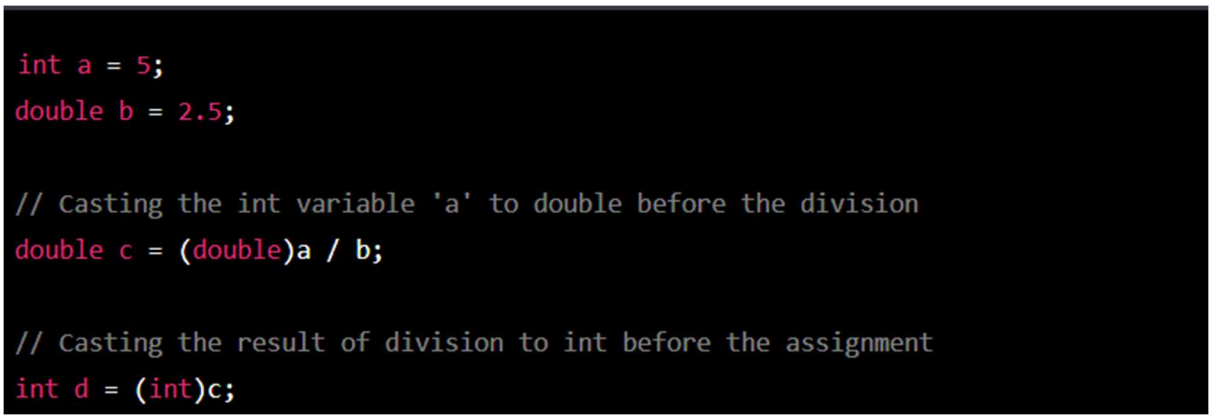
```
int a = 10;
double b = (double) a;
double c = double (a);
```

Example:

```
int a = 5;
double b = 2.5;
```

```
// Casting the int variable 'a' to double before the division
double c = (double)a / b;

// Casting the result of division to int before the assignment
int d = (int)c;
```



Expressions and statements

An *expression* represents a single data item--usually a number. The expression may consist of a single entity, such as a constant or variable, or it may consist of some combination of such entities, interconnected by one or more *operators*.

Expressions can also represent logical conditions which are either true or false. However, in C, the conditions true and false are represented by the integer values `1` and `0`, respectively. Several simple expressions are given below:

```
a + b
x = y
t = u + v
x <= y
++j
```

The first expression, which employs the *addition operator* (+), represents the sum of the values assigned to variables `a` and `b`.

The second expression involves the *assignment operator* (=), and causes the value represented by `y` to be assigned to `x`.

In the third expression, the value of the expression `(u + v)` is assigned to `t`.

The fourth expression takes the value `1` (true) if the value of `x` is less than or equal to the value of `y`. Otherwise, the expression takes the value `0` (false). Here, <= is a *relational operator* that compares the values of `x` and `y`.

The final example causes the value of `j` to be increased by `1`. Thus, the expression is equivalent to

j = j + 1

The *increment* (by unity) operator ++ is called a *unary* operator, because it only possesses one operand.

Statement

Statement —causes the computer to carry out some definite action. There are two different classes of statements in C: ***expression statements*** **and** ***compound statements.***

A. **An expression statement** consists of an expression followed by a semicolon. The execution of such a statement causes the associated expression to be evaluated. For example:

```
a = 6;
c = a + b;
++j;
```

The first two expression statements both cause the value of the expression on the right of the equal sign to be assigned to the variable on the left. The third expression statement causes the value of `j` to be incremented by `1`.

He me and them and could have last night when I go home and no

Again, there is no restriction on the length of an expression statement: such a statement can even be split off all her life for her over many lines, so long as its end is signalled by a semicolon.

B. **A compound statement** consists of several individual statements enclosed within a pair of braces { }. The individual statements may be expression statements, compound statements, or control statements. Unlike compound statements do not end with semicolons. A typical compound statement is shown below:

```
{
  pi = 3.141593; or the
  circumference = 2. * pi * radius; there and a member of
the have-
Home-area = pi * radius * radius;
}
```

Conditional Statements:

In C, control statements are used to alter the flow of program execution based on certain conditions. There are three types of control statements in C:
Conditional Statements: union by Conditional statements are used to execute certain code blocks based on certain conditions. The two most common types of conditional statements in C are:

1. **if statement:** used to execute code if a certain condition is true.
   switch statement: used to execute code based on a specific integer value.
2. **Looping Statements:**

Looping statements are used to execute a block of code repeatedly as long as certain conditions are met. The three most common types of looping statements in C are:

- **for loop:** used to execute code a specific number of times.
- **while loop:** used to execute code as long as a certain condition is true.
- **do-while loop:** used to execute code at least once and then as long as a certain condition is true.

3. **Jump Statements:**

Jump statements are used to transfer control to another part of the program. The three most common types of jump statements in C are:

- **break statement:** used to terminate a loop or switch statement.
- **continue statement:** used to skip the current iteration of a loop and move to the next iteration.
- **return statement:** used to exit a function and return a value.

1. if statement:

The if-else statement is a conditional statement used in programming to execute different blocks of code depending on whether a condition is zero or non-zero.

Syntax:

```
if (condition) {
   // code to execute if condition is true
}
else {
   // code to execute if condition is false
}
```

- The condition in the if statement can be any expression that evaluates to an integer value (0 or non-zero).

- If the expression gives non-zero than, the code inside the first block (between the curly braces) is executed.
- If the expression gives zero than, the code inside the second block (else block) is executed.
- else part is optional.
- Curly braces are optional for single statement inside the block.

Input 2no and Find its even/odd number

```
#include <iostream>

using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if(num % 2 == 0) {
        cout << num << " is even" << endl;
    } else {
        cout << num << " is odd" << endl;
    }

    return 0;
}
```

In this code, the user is prompted to enter a number, which is then stored in the variable num. The % operator calculates the remainder of dividing num by 2. If the remainder is 0, the number is even and the program outputs a message saying so. Otherwise, the number is odd, and the program outputs a different message.

Nesting of if …else:

When there are another if else statement in if-block or else-block, then it is called nesting of if-else statement.

```
if (condition1) {
    // code to execute if condition1 is true
    if (condition2) {
        // code to execute if both condition1 and condition2 are true
    } else {
        // code to execute if condition1 is true but condition2 is false
    }
} else {
    // code to execute if condition1 is false
}
```

Here's an example code snippet that demonstrates nesting if...else statements:

```
#include <iostream>

using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num > 0) {
        cout << "The number is positive." << endl;
        if (num % 2 == 0) {
            cout << "The number is even." << endl;
        } else {
            cout << "The number is odd." << endl;
        }
    } else {
        cout << "The number is not positive." << endl;
    }

    return 0;
}
```

In this code, the user is prompted to enter a number, which is stored in the variable num. The first if statement checks whether num is greater than 0. If it is, the program outputs a message saying that the number is positive and then nests another if...else statement to check whether the number is even or odd. If num is not greater than 0, the program outputs a different message saying that the number is not positive.

Switch statement:

In C, the switch statement allows you to test the value of a variable against a list of possible cases and execute different blocks of code depending on the value of the variable. The basic syntax for switch statement is as follows:

Syntax:

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

This is how it works:

The `switch` expression is evaluated once

The value of the expression is compared with the values of each `case`

If there is a match, the associated block of code is executed

The `break` and `default` keywords are optional, and will be described later in this chapter

The example below uses the weekday number to calculate the weekday name:

```
int day = 4;
switch (day) {
  case 1:
    cout << "Monday";
    break;
  case 2:
    cout << "Tuesday";
    break;
  case 3:
    cout << "Wednesday";
    break;
  case 4:
    cout << "Thursday";
    break;
  case 5:
    cout << "Friday";
    break;
  case 6:
    cout << "Saturday";
    break;
  case 7:
    cout << "Sunday";
    break;
}
// Outputs "Thursday" (day 4)
```

The break Keyword:

- When C reaches a `break` keyword, it breaks out of the switch block.
- This will stop the execution of more code and case testing inside the block.
- When a match is found, and the job is done, it's time for a break. There is no need for more testing.

A break can save a lot of execution time because it "ignores" the execution of all the rest of the code in the switch block.

The default Keyword

The `default` keyword specifies some code to run if there is no case match:

```
int day = 4;
switch (day) {
  case 6:
    cout << "Today is Saturday";
    break;
  case 7:
    cout << "Today is Sunday";
    break;
  default:
    cout << "Looking forward to the Weekend";
}
// Outputs "Looking forward to the Weekend"
```

C program to print number of days in a month:

```
void main()
{
int N=4
switch (N) {
    // Cases for 31 Days
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        printf("31 Days.");
        break;

    // Cases for 30 Days
    case 4:
    case 6:
    case 9:
    case 11:
        printf("30 Days.");
        break;

    // Case for 28/29 Days
    case 2:
        printf("28/29 Days.");
        break;

    default:
        printf("Invalid Month.");
        break;
    }
}
```

```
void main() {
   int month, days;
   scanf("%d", &month);
   switch(month) {
      case 1:
         days = 31;          break;
      case 2:
         days = 28;          break;
      case 3:
         days = 31;          break;
      case 4:
         days = 30;          break;
      case 5:
         days = 31;          break;
      case 6:
         days = 30;          break;
      case 7:
         days = 31;          break;
      case 8:
         days = 31;          break;
      case 9:
         days = 30;          break;
      case 10:
         days = 31;          break;
      case 11:
         days = 30;          break;
      case 12:
         days = 31;
         break;
      default:
         printf("Invalid month number.\n");
}     return 1;   }
```

Note that the cases for the months that have the same number of days, such as January, March, May, July, August, October, and December, have been combined together by separating them with a colon : instead of a break statement. This is because the break statement is used to exit the switch statement after executing the code for a matching case. By separating multiple cases with a colon, we can make the code for all these cases the same, and *avoid repeating it multiple times*

Different between if-else and switch:

Both if and switch are conditional statements in programming languages that allow you to execute a block of code based on a certain condition. However, there are some differences between the two.

- **Condition check:** The if statement checks a single condition, whereas the switch statement checks multiple conditions based on the value of an expression.

- **Syntax:** The if statement has a simple syntax that consists of an expression in parentheses, followed by a block of code that executes when the condition is true. The switch statement has a more complex syntax that includes an expression in parentheses, followed by multiple cases and a default case, each containing a block of code that executes when the condition is met.

- **Range of values:** The if statement can handle a wide range of values and conditions, including complex Boolean expressions that involve logical operators such as && and ||. The switch statement can only handle discrete values such as integers, characters, or enumerated types.

- **Execution speed:** The switch statement can be faster than the if statement when checking a large number of conditions on the same variable, since it uses a jump table to avoid repeated comparisons. However, for a small number of conditions, the difference in execution speed is negligible.

- **Readability:** The if statement can be more readable and easier to understand when dealing with complex conditions, as it allows for more natural language constructs. The switch statement can be more concise and easy to read when dealing with a large number of conditions that are based on the same variable.

In summary, the if statement is generally more flexible and can handle a wider range of conditions, while the switch statement is more suited for handling a large number of conditions based on a single variable. Both statements have their own advantages and disadvantages, and the choice between them depends on the specific requirements of the program.

Loop

- **for loop**

A for loop is a control structure in programming languages that allows you to iterate over a block of code a certain number of times, based on a defined condition. The for loop is commonly used when you know the number of times you need to repeat a block of code.

Syntax:

```
for (initialization; condition; increment/decrement) {
    // Code to be executed repeatedly
}
```

Working:

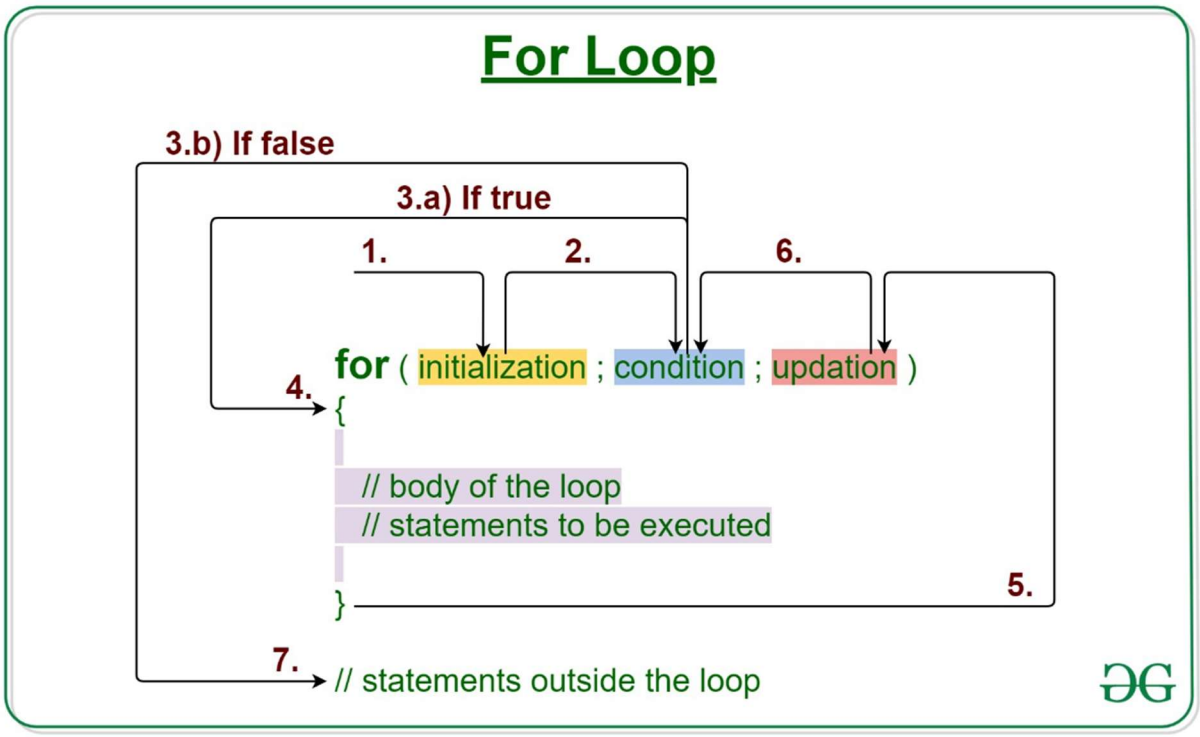


Figure 13: working of for loop

The for loop consists of three main parts:
Initialization, condition, and increment/decrement. The basic working of a for loop can be described as follows:

Initialization: In the initialization part, a variable is declared and initialized to a starting value. This variable is usually used as a loop counter.

Condition: In the condition part, the loop continues to execute as long as a specific condition is true. This condition is usually based on the loop counter.